Problem 2: Recurrent linear attention in Triton

GStar Bootcamp

Abstract

Implement a Triton decoding kernel using rank-one state writes and query reads.

Starter file: `problem_2.py`. **Checker:** `python autograder.py -p2`. **Reading path:** Overview, P1. Next: P3.

Mathematical background: an outer product is a memory write

P1 changed the attention rule; P2 changes only its execution. For one value coordinate a, the recurrent state column evolves as

$$(\mathbf{S}_t)_{:,a} = (\mathbf{S}_{t-1})_{:,a} + \boldsymbol{k}_t v_{ta}, \qquad o_{ta} = \boldsymbol{q}_t^\top (\mathbf{S}_t)_{:,a}.$$

Each column receives the same key direction but a different scalar value. Columns do not depend on one another. This algebra explains the supplied grid: one program can own a value-channel tile while retaining all key coordinates. The update is a rank-one matrix write, not a dot product; only the subsequent query read contracts the key axis.

For an intuitive read, write the carried memory as a sum of associations:

$$\mathbf{S}_t^\top \boldsymbol{q}_t = \mathbf{S}_0^\top \boldsymbol{q}_t + \sum_{j \le t} \boldsymbol{v}_j (\boldsymbol{k}_j^\top \boldsymbol{q}_t).$$

The current query selects a mixture of stored values through its alignment with their keys. This is the associative-memory interpretation behind the recurrence [1]. It does not recover an arbitrary value perfectly: correlations between keys mix associations, a limitation that motivates P5.

The program must update memory before reading it because the causal operator includes token t. Keeping state on chip reduces repeated state traffic, but the dependency from $\mathbf{S}_{t-1}$ to $\mathbf{S}_t$ remains sequential. P3 will move most prompt-processing work into chunk-local matrix products without changing this read/write timing.

Before coding. Identify which operation preserves both state axes, which operation reduces an axis, and why splitting the value axis needs no cross-program reduction.

Work assignment

Flatten batch and head into $p = bH + h$. A program owns one $(p, \text{value tile})$ pair. Its state tile has shape (D_k, B_V), with $B_V = 64$. Value tiles are disjoint, so programs can write outputs and final-state tiles without atomics.

For contiguous inputs, the address of a query element is

$$\text{offset}_Q(b, h, t, d) = ((bH + h)T + t)D_k + d, \tag{2.1}$$

using zero-based code indices. Value addresses replace D_k with D_v; state addresses are $((bH + h)D_k + i)D_v + j$.

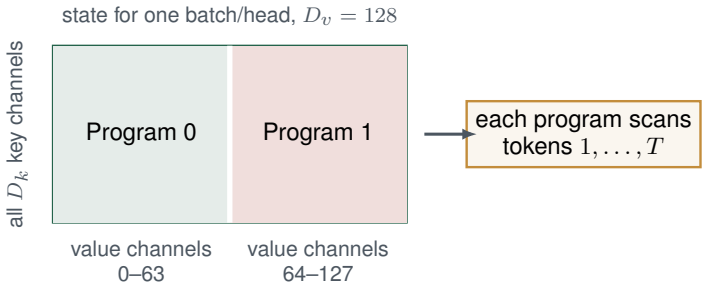


Figure 1. The recurrent kernel partitions the value axis. Each program retains its own state tile while scanning the sequence.

The arithmetic inside one token

After the launcher has loaded $\boldsymbol{q}, \boldsymbol{k} \in \mathbb{R}^{D_k}$ and a value tile $\boldsymbol{v} \in \mathbb{R}^{B_V}$, the program performs

$$S_{ij} \leftarrow S_{ij} + k_i v_j, \qquad o_j = \sum_{i=1}^{D_k} q_i S_{ij}.$$

The first operation broadcasts a key column against a value row. The second reduces the key dimension. Perform both in FP32; cast only the stored output to its requested dtype.

Provided infrastructure

`problem_2.py` provides kernel signatures, grids, pointer setup, input loads, the token loop, and stores. Complete the state update and output reduction in `_recurrent_forward`.

Problem 2: implementation steps and checks

1. **Update the state (30 points).** Broadcast the loaded key and value into a (D_k, B_V) outer product, then add it to the carried state. Do not replace the state with the current outer product.

2. **Read the output (25 points).** Multiply the updated state by the query column and reduce over the key dimension. The output tile has shape $(B_V,)$. Apply the code's `scale` exactly once.

3. **Return the final state and explain ownership (20 points).** Store the carried state after the final real token. Explain why distinct value tiles do not overlap.

```
value_block = tl.program_id(0)
batch_head = tl.program_id(1)
key_offsets = tl.arange(0, HEAD_DIM)
value_offsets = value_block * BLOCK_V + tl.arange(0, BLOCK_V)
# Pointer setup and initial-state load are provided.
for t in range(seq_len):
    q = tl.load(q_ptrs + t * HEAD_DIM).to(tl.float32) * scale
    k = tl.load(k_ptrs + t * HEAD_DIM).to(tl.float32)
    v = tl.load(v_ptrs + t * VALUE_DIM).to(tl.float32)
    # TODO P2.A: add the rank-one write into state.
    # TODO P2.B: read state and reduce along the key axis.
    tl.store(out_ptrs + t * VALUE_DIM, out.to(Out.dtype.element_ty))
# Final-state store is provided; it must store your updated state.
```

Table 1. Useful shape reasoning.

Expression role	Shape	Triton concept
Key column	$(D_k, 1)$	Insert a singleton value axis
Value row	$(1, B_V)$	Insert a singleton key axis
State/write	(D_k, B_V)	Elementwise broadcast arithmetic
Output read	$(B_V,)$	Reduction over axis 0

Useful shape reasoning

Verification

Compare against Problem 1 with $T = 1$ and with multiple tokens. Test $D_k/D_v = 128/64$ and $64/128$ to catch wrong reductions and missing value tiles. With a nonzero initial state and zero keys, every output must be the scaled initial-state read, and the final state must equal the input state.

Split a sequence at an interior token. Run the first segment, pass its final state to a second call, and compare the concatenated outputs with a single full call.

Understand the performance limit

This kernel is recurrent across tokens. It is useful for decoding and for a correctness baseline, but every token waits for the preceding state. Problem 3 moves much of the sequence work into chunk-local matrix products.

Problem 2: pointer layout and program ownership

Flattening batch and head

Let $f = bH + h$, where b and h are zero-based code indices. Contiguity makes the linear address of a query/key element

$$\text{offset}_Q(b, h, t, d) = fTD_k + tD_k + d.$$

For values and outputs, replace D_k with D_v. The address of state element (d, a) is

$$\text{offset}_S(b, h, d, a) = fD_kD_v + dD_v + a.$$

These offsets explain the supplied pointer bases. The value stride must use D_v, even when D_k and D_v happen to be equal in an initial test.

Why partition the value axis?

One recurrent program owns all D_k key rows and a disjoint value-channel interval of length $B_V = 64$. For $D_k = 128$ and $D_v = 128$, two programs independently carry state tiles of shape 128×64. Every prediction/read within one program reduces all 128 key coordinates. Each program writes 64 distinct output channels and 64 distinct state columns.

There is no cross-program communication: a key/value outer product can be partitioned across value columns, and a query read of one output channel depends only on its corresponding state column. Partitioning the key axis instead would require a reduction across programs to assemble each output channel, which this recurrent skeleton does not provide.

Broadcasting and reducing are different operations

A write preserves both axes:

$$\Delta S_{da} = k_d v_a \quad \Longrightarrow \quad \Delta \mathbf{S} \in \mathbb{R}^{D_k \times B_V}.$$

A read contracts the key axis:

$$o_a = \sum_d q_d S_{da} \quad \Longrightarrow \quad \boldsymbol{o} \in \mathbb{R}^{B_V}.$$

A reduction over axis 1 would produce D_k numbers and answer a different question. Test unequal dimensions early: equal-sized axes can hide this error.

Scale and cast placement

The actual starter loads `q` into FP32 and multiplies it by `scale` before the masked student region. Thus `q` in that region already means the scaled query. Do not multiply the reduction by the scale again. Keep state and intermediate products in FP32, then cast the stored output to the output pointer's element type. Store final memory in FP32.

Three diagnostic tests

With zero keys, memory stays equal to its initial value. With zero queries or zero scale, outputs vanish while memory still changes. With $T = 1$, the result must include the current outer product. Together these tests separate memory-update bugs from output-read bugs.

References

[1] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. *Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention*. 2020.
https://arxiv.org/abs/2006.16236